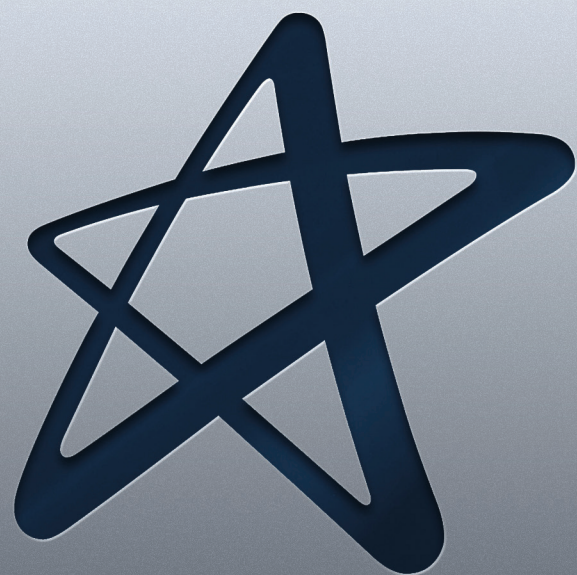


Computação Paralela e Distribuída



Cruzeiro do Sul Virtual
Educação a distância

Material Teórico



**Serviços de Nome, Tempo Global Coordenação
e Suporte de Sistemas Distribuídos**

Responsável pelo Conteúdo:

Prof. Esp. Allan Piter Pressi

Revisão Textual:

Prof.^a Dr.^a Selma Aparecida Cesarin

UNIDADE

Serviços de nome, Tempo Global Coordenação e Suporte de Sistemas Distribuídos



- Serviços de Nomes;
- Tempo e Estados Globais;
- Relógios, eventos e estados do Processo;
- Estados Globais;
- Coordenação e Acordo Entre Processos.



OBJETIVO DE APRENDIZADO

- Apresentar ao aluno os serviços de DNS, a importância da sincronização e o uso do relógio em Sistemas Distribuídos e a coordenação e comunicação de Processos.



Orientações de estudo

Para que o conteúdo desta Disciplina seja bem aproveitado e haja maior aplicabilidade na sua formação acadêmica e atuação profissional, siga algumas recomendações básicas:



Assim:

- ✓ Organize seus estudos de maneira que passem a fazer parte da sua rotina. Por exemplo, você poderá determinar um dia e horário fixos como seu “momento do estudo”;
- ✓ Procure se alimentar e se hidratar quando for estudar; lembre-se de que uma alimentação saudável pode proporcionar melhor aproveitamento do estudo;
- ✓ No material de cada Unidade, há leituras indicadas e, entre elas, artigos científicos, livros, vídeos e *sites* para aprofundar os conhecimentos adquiridos ao longo da Unidade. Além disso, você também encontrará sugestões de conteúdo extra no item **Material Complementar**, que ampliarão sua interpretação e auxiliarão no pleno entendimento dos temas abordados;
- ✓ Após o contato com o conteúdo proposto, participe dos debates mediados em fóruns de discussão, pois irão auxiliar a verificar o quanto você absorveu de conhecimento, além de propiciar o contato com seus colegas e tutores, o que se apresenta como rico espaço de troca de ideias e de aprendizagem.

Serviços de Nomes

Serviço de Nomes funciona como um serviço distinto, que é usado pelos clientes de uma Rede de Computadores para obter acesso aos recursos da rede por meio do nome do recurso, ao invés de utilizar seu endereçamento de Rede.

As entidades nomeadas podem ser de vários tipos e podem ser gerenciadas por serviços diferentes; por exemplo, os Serviços de Nomes costumam ser usados para conter os endereços e outros detalhes de usuários, computadores, domínios de Rede, serviços e objetos remotos.

Questões básicas de *design* para serviços de nomes, como a estrutura e o gerenciamento de espaço de nomes reconhecidos pelo Serviço e as operações que o Serviço de Nomes suporta são descritas e ilustradas no contexto do Sistema de Nome de Domínio da *Internet* (DNS).

Também examinamos como os Serviços de Nomes são implementados, cobrindo aspectos como navegação por uma coleção de servidores de nomes ao escolher um nome, nomeando Dados em cache e replicando os Dados de nomeação para aumentar o desempenho e a disponibilidade.

Introdução

Num Sistema Distribuído, os nomes são usados para se referir a uma ampla variedade de recursos, como computadores, serviços, objetos e arquivos remotos e, também, os usuários.

Nomes não são o único meio útil de identificação: os atributos descritivos são outros. Às vezes, os clientes não sabem o nome da entidade específica que procuram, mas eles têm alguma informação que a descreve ou podem exigir um serviço e conhecer algumas de suas características, mas não o que a entidade implementa.

Nomes, Endereços e outros Atributos

Qualquer Processo que requeira acesso a um recurso específico deve ter um nome ou um identificador para isso. Exemplos de nomes legíveis por humanos são nomes de arquivos, como “arquivo.doc”, URLs, como `<http://<www.cruzeirodosul.com.br>`, e nomes de domínio da Internet, como `<www.google.com>`.

O termo identificador é usado, às vezes, para se referir a nomes que são interpretados apenas por Programas.

Dizemos que um nome é resolvido quando é traduzida a identificação do recurso ou objeto no qual se necessita ou deseja executar uma ação, como, por exemplo, realizar a impressão de um documento, invocando seu serviço de impressão pelo nome do recurso.

A associação entre um nome e um objeto é chamada de ligação.

O **DNS** mapeia nomes de domínio para os atributos de um computador *host*: seu endereço **IP**, o tipo de entrada (por exemplo, uma referência a um servidor de correio ou outro *host*) e o período de tempo pelo qual a entrada do anfitrião permanecerá válida.

O serviço de diretório **X500** pode ser usado para mapear o nome de uma pessoa em atributos, incluindo seu endereço de e-mail e o número de telefone.

Observe que um “endereço” pode ser considerado apenas outro nome, que deve ser pesquisado ou pode conter esse nome. Um endereço IP deve ser procurado para obter uma rede de endereço, como um endereço Ethernet.

Da mesma forma, os navegadores da web e os clientes de *email* usam o DNS para interpretar os nomes de domínio em *URLs* e endereços de e-mail.

Muitos nomes usados num Sistema Distribuído são específicos para algum serviço particular; por exemplo, usuários do site de Rede Social *twitter.com* têm nomes como *@nome* que nenhum outro serviço resolve.

Um cliente também pode usar um nome específico ao solicitar a um serviço para executar uma operação num objeto ou num recurso nomeado que ele gerencia; por exemplo, um nome de arquivo é dado ao serviço de arquivo ao solicitar que ele seja excluído, e um identificador de Processo é apresentado ao serviço de Gerenciamento de Processos ao solicitar que seja enviado um sinal.

Esses nomes são usados apenas no contexto do serviço que gerencia os objetos nomeados, exceto quando os clientes se comunicam sobre objetos compartilhados.

Identificadores Uniformes de Recursos (*URLs*) surgiram da necessidade de identificar recursos na *Web* e outros recursos, como caixas de correio eletrônicas. Um objetivo importante foi identificar recursos de forma coerente, para que todos possam ser processados por um *software* comum, como navegadores, por exemplo.

URLs são “uniformes”, pois sua sintaxe incorpora a de muitos tipos individuais de identificadores de recursos (ou seja, esquemas de *URL*) e existem procedimentos para gerenciar o namespace global de esquemas.

A vantagem da uniformidade é que facilita o Processo de introdução de novos tipos de identificadores, bem como a utilização de tipos existentes de identificador em novos contextos, sem interromper o uso existente.

Localizadores Uniformes de Recursos

Alguns URLs contêm informações que podem ser usadas para localizar e acessar um recurso; outros são nomes de recursos puros.

O termo familiar *URL* é frequentemente usado para *URLs* que fornecem informações de localização e para especificar o método de acesso ao recurso, incluindo os URLs “**http**”, que identificam uma página da *Web* no caminho fornecido (‘/’).; outro exemplo é uma URL “*mailto*”, como *mailto: nome@empresa.com.br*, que identifica a caixa de correio no endereço fornecido.

Serviços de Nomes e o Sistema de Nomes de Domínio

Um Serviço de Nomes armazena informações sobre uma coleção de nomes textuais, na forma de ligações entre os nomes e os atributos das entidades que eles denotam, como usuários, computadores, serviços e objetos.

O Gerenciamento de Nomes é separado de outros serviços em grande parte devido à abertura de Sistemas Distribuídos, que trazem as seguintes motivações:

- **Unificação:** muitas vezes é conveniente usar recursos gerenciados por diferentes serviços no mesmo esquema de nomenclatura; *URIS* são um bom exemplo disso;
- **Integração:** nem sempre é possível prever o escopo do compartilhamento num Sistema; pode ser necessário compartilhar e, portanto, nomear recursos que foram criados em diferentes domínios administrativos.

Os serviços de nomes eram originalmente bastante simples; eles foram projetados apenas para atender à necessidade de vincular nomes a endereços num único domínio de gerenciamento, correspondente a uma única *LAN* ou *WAN*.

Espaços de Nomes

Um *namespace* é a coleção de todos os nomes válidos reconhecidos por um serviço específico. O serviço tentará procurar um nome válido, mesmo que esse nome possa não corresponder a qualquer objeto.

Os nomes podem ter uma estrutura interna que represente sua posição numa estrutura hierárquica: espaço de nomes, como nomes de caminho num Sistema de Arquivos ou numa hierarquia organizacional, como nomes de domínio da *Internet*; ou eles podem ser escolhidos de um conjunto plano de valores numéricos ou simbólicos identificadores.

No caso de Sistemas de Arquivos, cada diretório representa um contexto.

Assim, **/etc/passwd** é um nome hierárquico com dois componentes. O primeiro, “etc” é resolvido em relação ao contexto “/”, ou raiz, e a segunda parte, “passwd”, é relativa ao contexto “/etc”.

O nome **/oldetc/passwd** pode ter um significado diferente, porque seu segundo componente é resolvido num contexto diferente.

Da mesma forma, o mesmo nome **/etc/passwd** pode possuir diferentes contextos e/ou conteúdo em computadores diferentes.

A estrutura das *URLs* “http” e de seu espaço de nomes da *URL* também inclui nomes relativos, como, por exemplo “../img/fig1.jpg”.

Quando um navegador ou outro *web* cliente encontra um nome relativo, ele usa o recurso no qual o nome relativo é incorporado para determinar o nome do *host* do servidor e o diretório ao qual esse nome de caminho se refere.

Nomes *DNS* são *strings* chamados de nomes de domínio; alguns exemplos são <www.cruzeirosul.com.br> (um computador), br.com.

Os servidores *DNS* não reconhecem nomes relativos; todos os nomes são referenciados em um diretório raiz global. Um, “Alias”, é um tipo de apontamento semelhante a um *link* simbólico que permite ao serviço de *DNS* realizar uma associação rápida entre um endereço físico de um recurso (endereço *IP*) e o nome desse recurso.

Um domínio de nomeação é um espaço de nome para o qual existe uma única autoridade administrativa geral, responsável pela atribuição de nomes nele.

A responsabilidade por um domínio de nomenclatura, normalmente, anda de mãos dadas com a responsabilidade de gerir e manter atualizada a parte correspondente do Banco de Dados armazenada num servidor de nomes autoritativos e usados pelo Serviço de Nomes.

O *DNS* fornece um ambiente global e homogêneo de espaço de nome no qual um determinado nome se refere à mesma entidade, não importa qual Processo e qual computador o procura.

Resolução de Nomes

Para o caso comum de espaços de nomes hierárquicos, a resolução de nomes é uma interação iterativa ou Processo recursivo pelo qual um nome é repetidamente apresentado a contextos de nomeação para procurar os atributos a que se refere.

Para descobrir um nome dentro de uma rede de computadores ou entre diferentes sistemas, ele é apresentado, primeiramente, a algum contexto e a partir desse contexto são associados o nome do recurso e o caminho para se chegar à sua localização.

Qualquer serviço de nomes, como *DNS*, armazena um Banco de Dados e é usado por uma grande população de computadores e recursos que não irá armazenar todas as suas informações de nomes num único servidor, que seria um gargalo e um ponto crítico de falha.

Como o *DNS* é projetado para armazenar entradas para milhões de domínios e é acessado por inúmeros clientes, não seria possível ter todas as consultas iniciando num servidor raiz, mesmo se fosse replicado pesadamente.

O Sistema de Nomes de Domínio

O *Domain Name System* é um design de Serviço de Nomes cujo Banco de Dados de nomes é utilizado em toda a Internet. O *DNS* substituiu a nomenclatura original no surgimento da Internet, em que todos os nomes e endereços de host eram mantidos num único arquivo mestre central e baixados pelo *FTP* para todos os computadores que precisavam deles.

O *DNS* é projetado para uso em múltiplas implementações e cada uma pode ter seu próprio espaço de nome. Na prática, no entanto, apenas um é amplamente difundido e é esse o usado para nomear por meio da Internet. No nome *DNS* da Internet, o espaço é particionado tanto organizacionalmente, quanto de acordo com a geografia.

Uma lista completa de nomes de domínio genéricos atuais está disponível na Internet Atribuída Autoridade de Números [www.iana.org].

Os países costumam usar seus próprios subdomínios para distinguir suas Organizações.

O *DNS* da Internet é usado, principalmente, para a resolução simples de nomes de *host* e para procurar hosts de correio eletrônico, como descrito a seguir.

Em geral, os aplicativos usam o *DNS* para resolver nomes de *host* em endereços *IP*. Por exemplo, quando um navegador da Web recebe um *URL* contendo nome de domínio *www.cruzeirosul.com.br*, faz uma pergunta *DNS* e obtém o endereço *IP* correspondente.

Os navegadores usam *HTTP* para se comunicar com o servidor da web no endereço *IP* fornecido, usando uma porta reservada ao número, se nenhum for especificado no *URL*.

Os serviços *FTP* e *SMTP* funcionam de maneira semelhante: por exemplo, um programa de *FTP* pode receber o nome de domínio *ftp.cruzeirosul.com.br* e pode fazer uma pergunta *DNS* para obter o seu endereço *IP* e, em seguida, usar o *TCP* para se comunicar com ele no número da porta reservada.

O *software* de correio eletrônico usa o *DNS* para resolver nomes de domínio nos endereços *IP* dos anfitriões de correio, isto é, computadores que aceitarão correio para aqueles domínios.

Alguns outros tipos de consulta são implementados em algumas instalações, mas são menos frequentemente usados do que aqueles que são dados.

Alguns *softwares* requerem que um nome de domínio seja retornado Endereço de *IP*. Esse é apenas o contrário da consulta de nome de *host* normal, mas o servidor de nomes Receber consulta e responde somente se o endereço *IP* estiver em seu próprio domínio.

Para economizar em comunicação de Rede, o protocolo *DNS* permite várias consultas a serem empacotadas na mesma mensagem de solicitação para servidores de nomes correspondentes e envia várias respostas a cada solicitação.

O *Berkeley Internet Name Domain (BIND)* é uma implementação do *DNS* para computadores que executam o *UNIX*. O *BIND* permite três categorias de servidores de nomes: **servidores primários, secundários e servidores somente de armazenamento em cache.**

Serviços de Diretório

Descrevemos como os Serviços de Nome armazenam coleções de pares e como os atributos são pesquisados a partir de um nome. É natural considerar o dual desse arranjo, em que os atributos são usados como valores a serem pesquisados.

Nesses serviços, nomes textuais podem ser considerados apenas outro atributo. Às vezes, os usuários desejam encontrar uma pessoa ou um recurso em particular, mas eles não sabem seu nome; apenas alguns de seus outros atributos.

Um serviço que armazena coleções de ligações entre nomes e atributos e que procura entradas que correspondam a especificações baseadas em atributos é chamado de **Serviço de Diretório**.

Exemplos são os serviços do Active Directory da Microsoft, X.500 e seu primo *LDAP*. Os Serviços de Diretório, às vezes, são chamados de **Serviços de Páginas Amarelas** e o nome convencional Serviços corresponde aos chamados **Serviços de *white pages***, numa analogia com os tipos tradicionais de lista telefônica.

Os **Serviços de Diretório** também são conhecidos como **Serviços de Nomes Baseados em Atributos**.

Atributos são claramente mais poderosos que nomes como designadores de objetos: Programas podem ser escritos para selecionar objetos de acordo com especificações de atributos precisos em que nomes podem não ser conhecidos.

Resumo

Este item descreve a concepção e a implementação de Serviços de Nomes em Sistemas Distribuídos.

Serviços de Nome armazenam os atributos de objetos num Sistema Distribuído em particular, seus endereços – e retornam esses atributos quando um nome textual é fornecido.

Os principais requisitos para o Serviço de Nomes são: capacidade de lidar com um número de nomes, longa vida útil, alta disponibilidade, isolamento de falhas e tolerância de desconfiança.

A questão principal do *design* é a estrutura do espaço do nome – as regras sintáticas governando nomes. Uma questão relacionada é o modelo de resolução, que define as regras pelas quais o nome de multicomponentes é resolvido para um conjunto de atributos.

O conjunto de nomes ligados deve ser gerenciado. A maioria dos projetos considera o espaço do nome a ser dividido em domínios – seções discretas do *namespace*, cada uma delas associada a uma única autoridade controlando a ligação de nomes dentro dela.

A implementação do Serviço de Nomes pode abranger diferentes organizações e comunidades de usuários. A coleção de ligações entre nomes e atributos, em outras palavras, é armazenada em vários servidores de nomes, cada um dos quais armazena pelo menos parte do conjunto de nomes dentro de um domínio de nomenclatura.

A questão da navegação, portanto, surge: Por qual procedimento pode ser resolvida quando as informações necessárias são armazenadas em *sites*?

Os tipos de navegação suportados são **iterativos**, **multicast** e **recursivo controlado pelo servidor e não recursivo controlado pelo servidor**.

Outro aspecto importante da implementação de um Serviço de Nomes é o uso de replicação e de cache. Ambos ajudam a tornar o serviço altamente disponível, e ambos também reduzem o tempo gasto para localizar um nome.

Este item considerou dois casos principais de *design* de serviços de nomes e implementação.

O *Domain Name System* é amplamente usado para nomear computadores e endereçamento de correio eletrônico pela *Internet* e alcança bons tempos de resposta por meio de replicação e de cache.

O Serviço Global de Nomes é um *design* que abordou a questão de reconfigurar o *namespace* à medida que ocorrem mudanças organizacionais. O item também considerou serviços de diretório, que fornecem dados sobre objetos e serviços correspondentes quando os clientes fornecem descrições baseadas em atributos.

O X.500 é um modelo para serviços de diretório que pode variar no escopo de organizações individuais para diretórios globais; foi adotado mais amplamente para uso em *intranets*, com a chegada do *software LDAP*.

Tempo e Estados Globais

Neste item, apresentamos alguns tópicos relacionados à questão do tempo em Sistemas. O tempo é uma questão prática importante; por exemplo, exigimos computadores ao redor do mundo para registrar as transações de comércio eletrônico de forma consistente.

O tempo também é uma importante construção teórica na compreensão de como as execuções distribuídas se desdobram, mas o tempo é problemático em Sistemas Distribuídos.

Cada computador pode ter seu próprio relógio, mas os relógios normalmente não possuem a mesma precisão de tempo, e não podemos sincronizá-los perfeitamente.

A ausência de tempo físico global torna difícil descobrir o estado de distribuir programas à medida que eles são executados. Muitas vezes, precisamos saber qual é o Processo de estado A quando o Processo B está em certo estado, mas não podemos confiar em relógios físicos para saber o que é verdade ao mesmo tempo.

Introdução

Este item apresenta conceitos fundamentais relacionados ao monitoramento de Sistemas Distribuídos à medida que a sua execução se desenrola e ao cronograma dos eventos que ocorrem em suas execuções.

O tempo é uma questão importante e interessante em Sistemas Distribuídos, por várias razões. Primeiro, o tempo é algo que muitas vezes queremos medir com precisão.

A fim de saber a que hora do dia um evento determinado ocorreu num computador particular, é necessário sincronizar seu relógio com uma fonte de tempo externa autoritativa; por exemplo, uma transação de comércio eletrônico envolve eventos no computador de um comerciante e num Banco computador. É importante, para fins de auditoria, que esses eventos tenham registro de data e hora com precisão.

O tempo de medição pode ser problemático devido à existência de múltiplos quadros de referência. Einstein demonstrou, em sua Teoria Especial da Relatividade, as intrigantes consequências que se seguem à observação de que a velocidade da luz é constante para todos os observadores, independentemente da sua velocidade relativa.

Ele provou, entre outras coisas, que dois eventos que são julgados simultâneos num quadro de referência não são necessariamente simultâneos de acordo com observadores em outras estruturas de referência que estão se movendo em relação a ele; por exemplo, um observador na Terra e um observador viajando para longe da Terra, numa nave espacial, vão discordar sobre o intervalo de tempo entre eventos, quanto mais sua velocidade relativa aumentar.

A ordem relativa de dois eventos pode até ser revertida para dois observadores diferentes, mas isso não pode acontecer se um evento fizer com que o outro ocorra.

Nesse caso, o efeito físico segue a causa física para todos os observadores, embora o tempo decorrido entre causa e efeito possa variar.

Assim, provou-se que o tempo dos eventos físicos foi relativo para o observador, e a noção de Newton de tempo físico absoluto mostrou-se sem fundamento. Não há um relógio físico especial no universo para o qual possamos apelar quando queremos medir intervalos de tempo.

A noção de tempo físico também é problemática num Sistema Distribuído. Isso não é devido aos efeitos da relatividade especial, que são insignificantes ou inexistentes para computadores (a menos que se considerem computadores viajando em naves espaciais!).

Pelo contrário; o problema baseia-se numa limitação semelhante em nossa capacidade de registrar eventos de data e hora em diferentes nós com precisão suficiente para saber a ordem em que ocorreu qualquer par de eventos, ou se eles ocorreram simultaneamente.

Não há tempo global absoluto ao qual se possa apelar e, ainda assim, às vezes precisamos observar Sistemas Distribuídos e estabelecer se certos estados de coisas ocorreram ao mesmo tempo; por exemplo, em Sistemas Orientados a objetos, precisamos ser capazes de estabelecer se as referências a um determinado objeto não existem mais – se o objeto se tornou lixo (caso em que podemos libertar a memória).

Estabelecer isso requer observações dos estados dos Processos (para descobrir se contêm referências) e dos canais de comunicação entre os Processos (no caso de mensagens contendo referências estarem em trânsito).

Vamos examinar os métodos pelos quais os relógios de computador podem ser aproximadamente sincronizados, usando a passagem de mensagens.

Relógios, eventos e estados do Processo

A sincronização entre Processos é tão importante quanto a comunicação entre Processos em Sistemas Distribuídos. Por exemplo, como as regiões críticas são implementadas num Sistema Distribuído, e como esses recursos são alocados?

Em Sistemas de uma única *CPU*, regiões críticas, exclusão mútua e outros problemas de sincronização são geralmente resolvidos usando métodos como semáforos e monitores.

Esses métodos não são recomendados para serem usados em Sistemas Distribuídos porque eles, invariavelmente, contam (implicitamente) com a existência de uma memória compartilhada. Por exemplo, dois Processos que estão interagindo usando um semáforo: ambos devem ser capazes de acessar o semáforo.

Se estão rodando na mesma máquina, eles podem compartilhar o semáforo, tendo-o armazenado no *Kernel*, e executar Chamadas de Sistema (*System Calls*) para acessá-lo.

Se, no entanto, eles estiverem rodando em diferentes máquinas, esse método não mais funcionará e outras técnicas devem ser utilizadas. Mesmo parecendo um problema simples, como determinar se o evento A aconteceu antes ou depois do evento B, requer bastante cuidado.

Inicialmente, será focalizado o tempo e a forma de medi-lo, devido ao fato de que o tempo é a principal parte de alguns métodos de sincronização.

Em seguida, será vista a exclusão mútua e os algoritmos de eleições. Depois, será estudada uma técnica de sincronização de alto nível chamada de transação atômica. Finalmente, será estudado o *deadlock*.

Sincronização de Relógio

A sincronização em Sistemas Distribuídos é mais complicada do que em Sistemas Centralizados porque naqueles é necessário utilizar algoritmos distribuídos.

Não é usualmente possível (ou desejável) coletar todas as informações sobre o Sistema num único lugar, e depois deixar que alguns Processos as examinem e tomem uma decisão, como é feito no caso centralizado.

Em geral, algoritmos distribuídos possuem as seguintes propriedades:

1. A informação relevante está espalhada em múltiplas máquinas;
2. Processos tomam decisões baseadas somente nas informações locais;
3. Um único ponto de falha no Sistema deve ser evitado;
4. Não existe um relógio em comum ou outro tipo preciso de tempo global.

Relógios Lógicos

Quase todos os computadores possuem um circuito para se manter a par do tempo. O *timer* utilizado nos computadores é, geralmente, produzido a partir de cristal de quartzo. Quando mantido sob tensão, o cristal de quartzo oscila numa frequência bem definida, que depende do tipo de cristal, como é cortado e a quantidade de tensão.

Associados a cada cristal, existem dois registros, um **counter** e um **holding register**. Cada oscilação do cristal decrementa o counter por um.

Quando o *counter* chega a zero, uma interrupção é gerada e o *counter* é recarregado pelo *holding register*. Desse modo, é possível programar o *timer* para gerar uma interrupção 60 vezes por segundo, ou qualquer outra frequência desejada; cada interrupção é chamada *clock tick*.

Quando o Sistema é inicializado, ele normalmente pede ao operador para entrar com a data e a hora, que são, então, convertidas para o número de *ticks* depois da data conhecida e armazenadas na memória.

A todo *clock tick*, o serviço de interrupção adiciona uma unidade ao tempo armazenado na memória. Desse modo, o *software clock* é mantido atualizado.

O Protocolo de Horário da Rede

O *Network Time Protocol (NTP)* (MILLS, 1995) define uma arquitetura para um serviço de tempo e um Protocolo para distribuir informações de tempo pela *Internet*.

Os principais objetivos e características do *NTP* são os seguintes: fornecer um serviço que permita aos clientes, através da *Internet*, sincronizarem com precisão o relógio de seu computador com o padrão *UTC*, embora ocorram atrasos nessa sincronização de mensagem devido à comunicação através da *Internet*.

O *NTP* emprega técnicas estatísticas para a filtragem de dados de temporização e discrimina a qualidade dos dados de temporização de diferentes Servidores.

Estados Globais

Nesta e na próxima seção, examinaremos o problema de descobrir se uma determinada propriedade é verdadeiramente de um Sistema Distribuído quando ele é executado.

Começamos dando os exemplos de **coleta de lixo distribuída**, **detecção de deadlock**, **detecção de terminação** e **depuração**:

- **Coleta de lixo distribuída:** um objeto é considerado lixo se não houver mais quaisquer referências a ele em qualquer lugar do Sistema Distribuído;
- **Detecção de *deadlock* distribuída:** um *deadlock* distribuído ocorre quando um processo “A” aguarda até que outro processo envie uma mensagem comunicando que foi finalizado, e em que há um relacionamento de “espera” até a conclusão do processo que está interagindo com o processo “A”;
- **Detecção de terminação distribuída:** o problema aqui é como detectar que um algoritmo distribuído foi finalizado.

Os Sistemas Distribuídos são complexos para depurar e cuidados devem ser tomados no estabelecimento do que ocorreu durante a execução.

Estados Globais

É possível, em princípio, observar a sucessão de estados de um Processo individual, mas a questão de como determinar um estado global do Sistema – o estado da coleção de Processos – é muito mais difícil de resolver.

O problema essencial é a ausência de tempo global. Se todos os Processos tivessem relógios sincronizados, então, poderíamos concordar com um tempo em que cada Processo iria gravar seu estado – o resultado seria um estado global real do Sistema.

Da coleção de estados de Processo, poderíamos dizer, por exemplo, se os Processos estavam num impasse, mas não podemos alcançar a sincronização perfeita do relógio; assim, esse método não está disponível para nós.

Então, poderíamos perguntar se podemos montar um estado global significativo do local dos estados registrados em diferentes momentos reais.

Cada evento é uma ação interna do Processo: ou é o envio ou a recepção de uma mensagem por meio da comunicação de canais que conectam os Processos.

Um estado global consistente é aquele que corresponde a um corte consistente. Nós podemos caracterizar a execução de um Sistema Distribuído como uma série de transições entre estados globais do Sistema.

Em cada transição, precisamente, um evento ocorre em algum Processo único no Sistema. Esse evento é o envio de uma mensagem, o recebimento de uma mensagem ou um evento interno e se dois eventos ocorreram simultaneamente podemos considerar que eles ocorreram numa ordem definida – digamos, ordenada, de acordo com os identificadores de Processo.

Às vezes, podemos alterar a ordenação de eventos concorrentes dentro de uma linearização e derivar uma execução que ainda passa apenas por estados globais consistentes; por exemplo, se dois eventos sucessivos numa linearização são o recebimento de mensagens por dois Processos, então, podemos trocar a ordem desses dois eventos.

Detectar uma condição como *deadlock* ou rescisão equivale a avaliar um problema global predicado de estado.

Um predicado de estado global é uma função que mapeia a partir do conjunto de estados de Processos no Sistema para Verdadeiro e Falso.

Também observamos aqui duas outras noções relevantes para os predicados globais do estado: **segurança** e **vivacidade**.

Depuração Distribuída

Examinamos, agora, o problema de registrar o estado global de um Sistema para que possamos fazer declarações úteis sobre um estado transitório em oposição a um estado estável.

Um exemplo seria um Sistema Distribuído que controla um Sistema de Tubos numa fábrica sobre o qual estamos interessados em saber se todas as válvulas (por diferentes Processos) foram abertas em algum momento.

Em geral, não podemos observar os valores das variáveis ou os estados das válvulas, simultaneamente. O desafio é monitorar a execução do Sistema ao longo do tempo para capturar informações de “rastreamento”, ao invés de um único instantâneo, para que possamos estabelecer se a condição necessária de segurança pode ter sido violada.

Nenhuma observação de um estado global consistente nos permite concluir se um predicado já foi avaliado para “Verdadeiro” na execução real. No entanto, podemos estar interessados em saber se eles podem ter ocorrido, tanto quanto podemos dizer, observando a execução.

A noção “definitivamente” se aplica à execução real e não a uma corrida que extrapolamos a partir dele.

Pode parecer paradoxal para nós considerarmos o que aconteceu numa execução real; no entanto, é possível avaliar se era definitivamente verdadeiro, considerando todas as linearizações dos eventos observados.

Coletando o Estado de um Sistema

Os Processos observados em Sistemas enviam seu estado inicial para o monitor e, posteriormente, de tempos em tempos, em mensagens de estado.

O monitor registra mensagens de estado de cada Processo numa fila separada, para cada entrada.

A atividade de preparação e o envio de mensagens de estado podem atrasar a execução dos Processos observados, mas não interfere de outra forma. Assim, não há necessidade de enviar o estado, exceto inicialmente, e quando ele muda.

Existem duas otimizações para reduzir o tráfego de mensagens de estado para o monitor. Primeiro, o estado global predicado pode depender apenas de certas partes dos estados dos Processos; por exemplo, apenas nos estados de variáveis particulares. Assim, os Processos observados precisam apenas enviar o estado relevante para o monitor.

Em segundo lugar, eles só precisam enviar seu estado às vezes, quando o predicado Eu posso se tornar Verdadeiro ou deixar de ser Verdadeiro. Não faz sentido enviar alterações para o estado que não afetam o valor do predicado.

Resumo

Este item começou descrevendo a importância de cronometragem precisa para Sistemas. Em seguida, descreveu algoritmos para sincronizar relógios, apesar do desvio entre eles e a variabilidade de atrasos de mensagens entre computadores.

O grau de precisão de sincronização, que é praticamente obtido, atende a muitos, mas não é suficiente para determinar a ordenação de um erro arbitrário.

Os relógios podem ser definidos como contadores atualizados, de acordo com o ocorrido antes da relação entre eventos.

Os relógios vetoriais são uma melhoria. É possível determinar examinando os carimbos de data/hora de vetor se dois eventos são ordenados por terem acontecido antes e depois ou se são concorrentes.

Coordenação e Acordo Entre Processos

Vamos conhecer, a partir de agora, alguns tópicos e algoritmos relacionados à questão de como os Processos coordenam suas ações e concordam com valores compartilhados em Sistemas Distribuídos, apesar das falhas.

Introdução

Este item apresenta os conceitos de algoritmos cujos objetivos variam, mas que compartilham um objetivo fundamental em Sistemas Distribuídos: um conjunto de Processos para coordenar suas ações ou concordar com um ou mais valores; por exemplo, no caso de uma peça complexa de máquina, como uma nave espacial, é essencial que os computadores que a controlam concordem com as condições, como se a missão da espaçonave estivesse em andamento ou como se tivesse sido abortada.

Além disso, os computadores devem coordenar suas ações corretamente com respeito a recursos compartilhados.

Uma distinção importante para nós será se a distribuição é assíncrona ou síncrona.

Num Sistema Assíncrono, podemos fazer suposições de tempo, e num Sistema Síncrono, vamos supor que existem limitações do tempo máximo de transmissão da mensagem, o tempo gasto para executar cada etapa de um Processo e as taxas de desvio do relógio.

As suposições síncronas nos permitem usar tempos limites para detectar falhas no Processo.

Pressupostos de Falha e Detectores de Falha

Por uma questão de simplicidade, este item assume que cada par de Processos está conectado por canais confiáveis, isto é, embora os componentes de Redes subjacentes possam sofrer falhas, os Processos usam um Protocolo de comunicação confiável que mascara essas falhas; por exemplo, retransmitindo mensagens ausentes ou corrompidas.

Também por causa de simplicidade, assumimos que nenhuma falha de Processo implica ameaça aos outros Processos.

Capacidade de comunicar: isso significa que nenhum dos Processos depende de outro para encaminhar mensagens.

Observe que um canal confiável acaba entregando uma mensagem para o destinatário. Num Sistema Síncrono, supomos que há redundância de *hardware* onde for necessário, de modo que um canal confiável não apenas entrega cada mensagem falha subjacente, mas o faz dentro de um limite de tempo especificado.

Em qualquer intervalo de tempo, a comunicação entre alguns Processos pode ter sucesso, enquanto a comunicação entre outros é atrasada; por exemplo, o fracasso de um roteador entre duas Redes pode significar que uma coleção de quatro Processos é dividida em dois pares, de modo que a comunicação intrapar seja possível em suas respectivas redes.

Qualquer que seja o tipo de falha, um Processo correto é aquele que não apresenta falhas em nenhum ponto na execução em consideração.

Observe que a correção se aplica ao todo da execução, não apenas para uma parte dela. Portanto, um Processo que sofre uma falha de falha é “sem falha” antes desse ponto; não “correto” antes desse ponto.

Uma falha não é necessariamente precisa. A maioria cai na categoria de detectores de falha não confiáveis. Um detector de falhas não confiável pode produzir um de dois valores dada à identidade de um Processo: **não suspeito** ou **suspeito**.

É importante perceber que, embora falemos de um detector de falhas atuando numa coleção de Processos, a resposta que o detector de falhas dá a um Processo é apenas tão boa quanto a informação disponível nesse Processo.

Um detector de falhas pode, às vezes, dar respostas diferentes a diferentes Processos, vez que as condições de comunicação variam de Processo para Processo.

Exclusão Mútua Distribuída

Os Processos distribuídos, geralmente, precisam coordenar suas atividades. Se uma coleção de Processos compartilha um recurso ou uma coleção de recursos, então, muitas vezes, a exclusão mútua é necessária para evitar interferências e garantir consistência ao acessar os recursos.

Num Sistema Distribuído, no entanto, nem as variáveis compartilhadas, nem as facilidades fornecidas por um único *kernel* local podem ser usadas para resolvê-lo, em geral.

Nós exigimos uma solução para distribuição e exclusão mútua: aquela que se baseia unicamente na passagem de mensagens.

Coordenação e acordo na Comunicação do Grupo

Vamos examinar os principais problemas de coordenação e acordo relacionados ao Grupo Comunicação, isto é, como alcançar a confiabilidade desejada e as propriedades de um pedido em todos os membros de um grupo durante a comunicação.

Os Processos podem enviar mensagens para um grupo. Essa mensagem é propagada para todos os membros do grupo com garantias em termos de confiabilidade e pedidos. Particularmente, estamos buscando a confiabilidade em termos das propriedades de validade, integridade e concordância; e ordenação em termos de ordenação.

O Sistema sob consideração contém uma coleção de Processos, que pode se comunicar de forma confiável em canais um para um.

Os Processos são membros de grupos, que são os destinos das mensagens enviadas com a operação *multicast*. Geralmente, é útil permitir que os Processos sejam membros de vários grupos simultaneamente, por exemplo, para permitir que Processos recebam informações de várias fontes, juntando vários grupos; mas, para simplificar nossa discussão sobre ordenar propriedades nós, às vezes, restringimos Processos para que sejam membros de, no máximo, um grupo de cada vez.

Cada mensagem transporta o identificador único do remetente do Processo que o enviou e o grupo de identificadores de grupo de destino exclusivo.

Multicast

O *Multicast* (também referido como *Multicast IP*) é, muitas vezes, usado para se referir a um “*broadcast* multiplexado”.

Multicast é a transmissão de informação para múltiplos destinatários, simultaneamente, usando a estratégia mais eficiente, em que as mensagens só passam por um link uma vez, e somente são duplicadas quando o link para os destinatários se divide em duas direções.

Em comparação ao *Multicast*, a entrega simples, ponto a ponto, é chamada de *Unicast*, e a entrega para todos os pontos de uma Rede chama-se *Broadcast*.

A palavra *Multicast* é tipicamente associada a *Multicast IP*, que é um Protocolo que transmite eficientemente pacotes para múltiplos pontos distintos, ao mesmo tempo, em Redes TCP/IP, usando um endereço *Multicast*. É comumente associado a aplicações de áudio/vídeo; por exemplo, Protocolo *RTP*.

Mas existem outros *Protocolos* na *Internet* que implementam o conceito *Multicast*. O *ATM*, por exemplo, possui mecanismos para conexões ponto para multiponto ou multiponto para multiponto.

Esse modelo, geralmente, assume que as estações participantes de uma comunicação sejam conhecidas com antecedência, de modo que árvores de distribuição possam ser geradas e recursos possam ser alocados pelos elementos da Rede.

Apesar de o *IP* ter um modelo conceitual bastante convincente, ele demanda muito mais recursos, equipamentos e processamento na Rede do que o modelo *Unicast best effort* ponto a ponto, o que tem gerado muitas críticas; porém, ainda não foi apresentado nenhum mecanismo que permita ao modelo de *Multicast IP* ser aplicado a uma escala de milhões de pontos e/ou milhões de grupos multicast, como seria de fato necessário para que as aplicações multicast em geral se difundissem na Internet comercial.

Até 2003, a maioria dos esforços para escalonar o *multicast* para grandes Redes tem se concentrado no simples caso no qual temos uma única fonte multicast, o que parece ser mais “tratável”, “computacionalmente” falando.

Por essa razão, e por motivos econômicos, o *Multicast IP* não está muito em uso na Internet comercial. Outras tecnologias *Multicast*, que não são baseadas no *Multicast IP*, são bem populares, tais como o *Internet Relay Chat*. Elas podem não ser tão elegantes como o *Multicast IP*, mas são pragmáticas e funcionam melhor para pequenos grupos.

Entretanto, algumas comunidades dentro da Internet pública fazem uso regular do *Multicast IP* (pesquise a *Mbone*, por exemplo), sendo, também, muito utilizado em aplicações especiais em Redes *IP* Privadas e na Internet 2 – a RNP é um exemplo disso no Brasil.

Multicast local, em que pacotes são enviados para grupos de *hosts* no mesmo *Data Link Layer* físico ou virtual, não requer roteamento muito complexo e é, portanto, muito mais utilizado.

Usa-se, por exemplo, no *IPv6*, para a resolução de nomes e endereços, e em redes *zeroconf*, para descobrir serviços, resolução de nomes e resolução de conflitos de endereços, substituindo Protocolos *Broadcast* ineficientes.

A segurança no *Multicast* é um dos maiores problemas. Soluções de comunicação segura comuns, geralmente, empregam criptografia simétrica, mas aplicá-las ao tráfego *Multicast IP* permitiria a qualquer um dos destinatários *Multicast* posar como o remetente.

Modelo do Sistema e Definições de Problemas

Se falhas arbitrárias podem ocorrer, outro fator na especificação do nosso Sistema é se os Processos assinam digitalmente as mensagens que eles enviam; e se Processos assinam suas mensagens, então, um Processo defeituoso é limitado no dano que ele pode fazer.

Especificamente, durante um algoritmo de acordo, ele não pode fazer uma declaração falsa sobre os valores que um Processo correto enviou para ele.

A relevância da assinatura de mensagens irá tornar-se mais clara quando discutirmos soluções para o Problema dos Generais Bizantinos; por padrão, assumimos que a assinatura não ocorre.

Para ajudar a entender como a formulação do problema se traduz num algoritmo, considere um Sistema no qual os Processos não podem falhar.

Note que a questão é apenas uma função possível e que os Processos poderiam concordar com um valor definido, por exemplo, se os valores forem ordenados, então, as funções mínima e máxima podem ser definidas.

Se os Processos podem falhar, isso introduz a complicação de detectar falhas e não é imediatamente claro que uma execução do algoritmo de consenso pode terminar. De fato, se o Sistema é assíncrono, então não pode. Voltaremos a esse ponto em breve.

Se os Processos podem falhar de forma arbitrária, então, Processos defeituosos podem, a princípio, comunicar valores aleatórios para os outros. Isso pode parecer improvável na prática, mas não está além dos limites da possibilidade um Processo com um *bug* falhar dessa maneira.

Uma falha poderia fazer um Processo enviar valores diferentes para diferentes recursos, numa tentativa de fazer com que os outros Processos que estão tentando chegar a um consenso falhem com essa questão.

No caso de inconsistência, os Processos corretos devem comparar o que eles receberam ao que os Processos alegam ter recebido.

O Problema dos Generais Bizantinos

No problema da Declaração Informal dos Generais Bizantinos (LAMPORT *et al.*, 1982), três ou mais generais devem concordar em atacar ou recuar.

Um, o comandante, emite a ordem. Os outros, tenentes do comandante, devem decidir se devem atacar ou recuar. Mas um ou mais generais podem ser “traíçoeiros”, isto é, com defeito.

Se o comandante é traíçoeiro, ele propõe a um general atacar e a outro recuar. Se um tenente é traíçoeiro, ele diz a um de seus colegas que o comandante disse-lhe para atacar e para outro que eles estão recuando.

Note que, para o Problema dos Generais Bizantinos, integridade implica concordância quando o comandante está correto; mas o comandante não precisa estar correto.

O problema de consistência interativa é outra variante do consenso, em que todo Processo propõe um valor único. O objetivo do algoritmo é que os Processos corretos concordem com um vetor de valores, um para cada Processo.

Nós chamamos esse vetor de **vetor de decisão**; por exemplo, o objetivo poderia ser cada um dos conjuntos de Processos obter as mesmas informações sobre seus respectivos estados.

Embora seja comum considerar que o Problema dos Generais Bizantinos tenha falhas arbitrárias de Processo, de fato, cada um dos três problemas – **consenso, generais bizantinos e consistência interativa** – é significativo no contexto de falhas arbitrárias ou de colisão. Da mesma forma, cada um pode ser enquadrado assumindo um Sistema Síncrono ou Assíncrono.

Às vezes, é possível obter uma solução para um problema usando uma solução para outro. Essa é uma propriedade muito útil, tanto porque aumenta nossa compreensão dos problemas, quanto porque, ao reutilizarmos as soluções, podemos potencialmente economizar esforço de implementação e de complexidade.

Em Sistemas com falhas de colisão, o consenso é equivalente a resolver problemas confiáveis e *Multicast* totalmente ordenado: dada uma solução para um, podemos resolver o outro.

O Problema dos Generais Bizantinos num Sistema Síncrono

Agora, discutiremos o Problema dos Generais Bizantinos num Sistema Síncrono. Um Processo defeituoso pode enviar qualquer mensagem, com qualquer valor, a qualquer momento; e pode omitir o envio de qualquer mensagem.

Processos corretos podem detectar a ausência de uma mensagem num tempo limite; mas não podem concluir que o remetente caiu, pois pode ser silencioso por algum tempo e depois enviar mensagens novamente.

Assumimos que os canais de comunicação entre pares de Processos são privados. Se um Processo pudesse examinar todas as mensagens que outros Processos enviaram, então, poderia detectar as inconsistências que um Processo defeituoso envia para diferentes Processos.

O Acordo Bizantino pode ser alcançado por três generais, com um deles com defeito, se os generais assinam digitalmente suas mensagens.

Como o algoritmo que os Processos executam é considerado correto, a simulação termina.

Os generais simulados corretos (nos dois Processos corretos) concordam e satisfazem a propriedade de integridade, mas agora nós temos um meio para os dois Processos corretos chegarem a um consenso: cada um decide sobre o valor escolhido por todos os seus generais simulados. Isso contradiz nosso resultado de impossibilidade dos Processos com um defeito.

Se um Processo correto não receber uma mensagem dentro de um prazo adequado (o Sistema é Síncrono), procederá como se o Processo defeituoso tivesse enviado o valor A.

Discussão

Podemos medir a eficiência de uma solução para o Problema dos Generais Bizantinos – ou qualquer outro problema semelhante – perguntando:

- Quantas rodadas de mensagens são necessárias?;
- Quantas mensagens são enviadas e de que tamanho?

Impossibilidade em Sistemas Assíncronos

Nós fornecemos soluções para o consenso e o Problema dos Generais Bizantinos (e daí, por derivação, a consistência interativa).

No entanto, em todas essas soluções, no Sistema Síncrono, os algoritmos assumem que as trocas de mensagens ocorrem em rodadas, e esses Processos têm o direito de expirar e assumir que um Processo defeituoso não enviou uma mensagem dentro da rodada, porque o atraso máximo de tempo foi excedido.

Num Sistema Assíncrono, os Processos podem responder às mensagens em momentos arbitrários; portanto, um Processo travado é indistinguível de um processo lento.

Essa abordagem não será analisada neste momento.

No entanto, existem outras técnicas para trabalhar em torno do resultado da impossibilidade, tais como: **mascaramento de falhas, detectores de falhas e comportamento dos Processos**.

A primeira técnica é evitar completamente o resultado da impossibilidade, mascarando qualquer falha de Processo que ocorra; por exemplo, os Sistemas de Transação empregam armazenamento persistente, que sobrevive a falhas de travamento.

Se um Processo falhar, ele será reiniciado. O Processo coloca informações suficientes em armazenamento persistente, em pontos, em seu Programa, para que, se ele falhar e for reiniciado, encontre dados suficientes para poder continuar corretamente com sua tarefa interrompida.

Em outras palavras, vai se comportar como um Processo que está correto, mas que, às vezes, leva muito tempo para executar uma etapa de processamento.

Outro método para contornar o resultado da impossibilidade é empregar detectores de falhas. Alguns sistemas práticos empregam detectores de falhas “perfeitos por Projeto”, para chegar a um consenso.

Nenhum detector de falha, num Sistema Assíncrono que funciona somente com a passagem de mensagens, pode ser realmente perfeito.

Um Processo que não responde não significa necessariamente que tenha falhado, mas os Processos restantes podem entender como se isso tivesse ocorrido. Eles fazem o que pode ser denominado “falha silenciosa”, por descartar quaisquer mensagens subsequentes que elas de fato recebam de um Processo “com falha”.

Esse método baseia-se no fato de o detector de falhas geralmente ser exato. Quando é impreciso, o Sistema deve prosseguir sem um membro do grupo que, de outra forma, poderia, potencialmente, contribuir para a eficácia do Sistema.

Resumo

Começamos este item discutindo a necessidade de Processos para acessar recursos compartilhados em condições de exclusão mútua.

Bloqueios nem sempre são implementados pelos servidores que gerenciam os recursos compartilhados.

Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta Unidade:



Sites

Como funciona o *DNS*

<https://goo.gl/hnuUpe>

Administração de Servidores

<https://goo.gl/eKKCG1>



Vídeos

A Importância do *DNS*

<https://youtu.be/epWv0-eqRMw>

Comunicação entre Processos

<https://youtu.be/uPMaNFIBXzI>

Referências

GAGLIARDI, Gary. **Cliente/servidor**. São Paulo: Makron Books do Brasil, 1996.

LAMPORT, L.; SHOSTAK, R.; PEASE, M. The Byzantine generals problem. ACM Trans, **Programming Lang Systems**, v.4, p. 382-401, 1982.

MILLS, D. **Simple Network Time Protocol**. Disponível em: <<https://tools.ietf.org/html/rfc1769>>.

STALLINGS, William. **Arquitetura e organização de computadores**: projeto para o desempenho. 8.ed. São Paulo: Pearson Prentice Hall, 2009.

TANENBAUM, Andrew S.; STEEN, Maarten van. **Sistemas Distribuídos: princípios e paradigmas**. 2.ed. São Paulo: Pearson, 2007.



Cruzeiro do Sul
Educatonal